

Factorisation de Bunch-Parlett et Bunch-Kaufman

MTH8211 - Projet phase 1

Hortense Beaudoin et Téo Dumoutier

Section 1 . Description du projet

Section 2 . Théorie

Section 3 . Implémentation

Section 4 . Tests et performance

Section 5 . Discussion

Section 6 . Retour sur les objectifs

1 Description du projet

Si on tente de résoudre un système $Ax = b$ où A est dense, plusieurs solutions existent, mais une majorité sont très coûteuses. La décomposition QR et la SVD existent pour toute matrice A , y compris les matrices indéfinies (i.e il existe x t.q $x^T Ax \leq 0$ et il existe y t.q $y^T Ay \geq 0$), mais elles n'exploitent pas forcément la structure du système à résoudre.

Certaines structures symétriques sont fréquemment rencontrées lors de la résolution de problèmes. On a par exemple vu en classe qu'une autre manière d'exprimer le problème de moindres carrés est de résoudre plutôt le système de point de selle

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} r \\ x \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}$$

où la matrice formée de A et A^T est symétrique et toujours indéfinie. Sous cette forme, le bloc supérieur gauche est bien évidemment inversible, puisqu'il est égal à l'identité. De manière intéressante, toute autre matrice B symétrique et indéfinie peut être permutée pour adopter une forme où le bloc supérieur gauche est inversible, soit

$$PBP^T = \begin{bmatrix} E & C^T \\ C & D \end{bmatrix}$$

où $D = D^T$ et $E = E^T$ inversible. Ce type de problème est donc particulièrement fréquent et il est intéressant de s'intéresser à une méthode de résolution où la symétrie des matrices est exploitée.

La décomposition de Cholesky n'existe pas pour ce genre de système puisque la matrice considérée n'est pas définie-positive, ce qui est une exigence associée à ce type de factorisation. Une factorisation LDL^* pourrait exister, mais pas systématiquement et elle court le risque d'être numériquement instable. Higham illustre bien ce risque dans son livre avec l'exemple suivant:

$$A = \begin{bmatrix} \epsilon & 1 \\ 1 & \epsilon \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1/\epsilon & 1 \end{bmatrix} \begin{bmatrix} \epsilon & 0 \\ 0 & \epsilon - 1/\epsilon \end{bmatrix} \begin{bmatrix} 1 & 1/\epsilon \\ 0 & 1 \end{bmatrix}$$

A est symétrique et si $\epsilon < 1$, indéfinie. On remarque que dans le cas où $\epsilon = 0$ la factorisation n'existe tout simplement et que le reste du temps, les facteurs $1/\epsilon$ risquent de grossir de manière disproportionnée, menant à un système mal conditionné ou à des erreurs numériques importantes.

Ce projet va donc plutôt s'intéresser aux factorisations de Bunch-Parlett et Bunch-Kaufman, qui exploitent toutes deux l'idée de permutation mentionnée précédemment. La mise en place algébrique étant la même dans les deux cas, nous allons dès maintenant la présenter brièvement.

2 Théorie

2.1 Factorisation en bloc pour matrices symétriques

Tel que mentionné ci-dessus, toute matrice $A \in \mathbb{C}^{n \times n}$ symétrique non-nulle peut être permutée de la manière suivante:

$$PAP^T = \begin{bmatrix} E & C^T \\ C & D \end{bmatrix}$$

où $E \in \mathbb{C}^{s \times s}$, $C \in \mathbb{C}^{n-s \times s}$ et $D \in \mathbb{C}^{(n-s) \times (n-s)}$ avec $s \in 1, 2$ et E inversible. Sous cette forme, il est possible de procéder à la factorisation suivante:

$$PAP^T = \begin{bmatrix} I_s & 0 \\ CE^{-1} & I_{n-s} \end{bmatrix} \begin{bmatrix} E & 0 \\ 0 & D - CE^{-1}C^T \end{bmatrix} \begin{bmatrix} I_s & E^{-1}C^T \\ 0 & I_{n-s} \end{bmatrix}$$

On reconnaît $D - CE^{-1}C^T$ comme étant le complément de Schur de E dans la matrice PAP^T . Ce complément est aussi symétrique, par symétrie des blocs E et D ;

$$(D - CE^{-1}C^T)^T = D^T - CE^{-T}C^T = D - CE^{-1}C^T$$

On peut donc répéter le processus sur cette matrice de dimension $(n-s) \times (n-s)$, puis de nouveau et ainsi de suite jusqu'à l'obtention de la factorisation complète.

À titre d'illustration, imaginons être à l'itération k avec la forme suivante;

$$P_k A_k P_k^T = \begin{bmatrix} E_k & C_k^T \\ C_k & D_k \end{bmatrix} = \begin{bmatrix} I_{s_k} & 0 \\ C_k E_k^{-1} & I_{n-s_k} \end{bmatrix} \begin{bmatrix} E_k & 0 \\ 0 & D_k - C_k E_k^{-1} C_k^T \end{bmatrix} \begin{bmatrix} I_{s_k} & E_k^{-1} C_k^T \\ 0 & I_{n-s_k} \end{bmatrix}$$

On pose alors $A_{k+1} = D_k - C_k E_k^{-1} C_k^T$ et on procède de la même manière.

$$P_{k+1} A_{k+1} P_{k+1}^T = \begin{bmatrix} E_{k+1} & C_{k+1}^T \\ C_{k+1} & D_{k+1} \end{bmatrix} = \begin{bmatrix} I_{s_{k+1}} & 0 \\ C_{k+1} E_{k+1}^{-1} & I_{n-(s_k+s_{k+1})} \end{bmatrix} \begin{bmatrix} E_{k+1} & 0 \\ 0 & D_{k+1} - C_{k+1} E_{k+1}^{-1} C_{k+1}^T \end{bmatrix} \begin{bmatrix} I_{s_{k+1}} & E_{k+1}^{-1} C_{k+1}^T \\ 0 & I_{n-(s_k+s_{k+1})} \end{bmatrix}$$

Et ainsi de suite. Les matrices P , L et B sont construites de la façon suivante à chaque itération:

$$P_{k+1} = P_{k+1} P_k \dots P_1$$

$$L_{k+1} = \begin{bmatrix} I_{s_k} & 0 & 0 \\ (C_k E_k^{-1})[1 : s_{k+1}, 1 : s_{k+1}] & I_{s_{k+1}} & 0 \\ (C_k E_k^{-1})[s_{k+1} + 1 : n, 1 : s_{k+1}] & (C_{k+1} E_{k+1}^{-1}) & I_{n-(s_k+s_{k+1})} \end{bmatrix}$$

$$B_{k+1} = \begin{bmatrix} E_k & 0 & 0 \\ 0 & E_{k+1} & 0 \\ 0 & 0 & A_{k+2} \end{bmatrix}$$

La différence entre les processus de Bunch-Parlett et Bunch-Kaufman repose dans le choix du pivot, soit de la permutation P_k . Le tout sera exploré plus en détail dans les prochaines sections.

2.2 Bunch-Parlett

La stratégie de Bunch-Parlett pour le choix du pivot est aussi appelée *pivotage complet*. Elle se base sur les valeurs maximales des entrées de la matrice sur et hors diagonal. Le choix du pivot est effectué comme suit;

Poser $\alpha = (1 + \sqrt{17})/8$

Poser $\mu_0 = \max_{i,j} |a_{ij}| =: |a_{pq}|$, ($q \geq p$)

Poser $\mu_1 = \max_i |a_{ii}| =: |a_{rr}|$

Si $\mu_1 \geq \alpha\mu_0$

Utiliser a_{rr} comme pivot 1×1 ($s = 1$, les colonnes et rangées 1 et r sont permutées)

Sinon

Utiliser $\begin{bmatrix} a_{pp} & a_{qp} \\ a_{qp} & a_{qq} \end{bmatrix}$ comme pivot 2×2 ($s = 2$, les colonnes et rangées 1 et p sont permutées, suivies des colonnes et rangées 2 et q)

On peut visualiser l'emplacement des pivots comme suit;

$$\begin{pmatrix} \ddots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \vdots & a_{rr} & & & & & \\ \vdots & & \ddots & & & & \\ \vdots & & & a_{pp} & \cdots & a_{qp} & \\ \vdots & & & \vdots & \ddots & \vdots & \\ \vdots & & & a_{qp} & \cdots & a_{qq} & \\ \vdots & & & & & & \ddots \end{pmatrix}$$

Il est intéressant de regarder de plus près la valeur de α et la manière dont cette dernière est obtenue. Elle est dérivée directement de la borne sur les éléments des blocs de B , elle-même issue des valeurs maximales contenues dans A . Comme cette valeur est reprise dans les processus de Bunch-Kaufman ainsi que dans l'alternative du *rook pivoting*, nous trouvons pertinent d'en présenter la brève dérivation ici. À noter que la dérivation s'effectue de manière semblable pour les deux autres processus et mène au même résultat.

2.2.1 Dérivation de α

Dans le cas du pivot 1×1 , ce dernier est choisi si $\mu_1 \geq \alpha\mu_0$, i.e $\max_i |a_{ii}| \geq \alpha \max_{ij} |a_{ij}|$. Toutes les mises à jour de A via le complément de Schur sont ainsi bornée;

$$|a_{ij}| = |b_{ij} - c_{i1} \frac{1}{e_{11}} c_{1j}| \leq \mu_0 + \frac{\mu_0^2}{\mu_1} \leq \left(1 + \frac{1}{\alpha}\right) \mu_0$$

On peut borner les valeurs de $|a_{ij}|$ de manière semblable pour le cas du pivot 2×2 . On note cette fois le complément de Schur comme

$$a_{ij} = b_{ij} - \begin{bmatrix} c_{i1} & c_{i2} \end{bmatrix} E^{-1} \begin{bmatrix} c_{11} \\ c_{21} \end{bmatrix}$$

où

$$E^{-1} = \frac{1}{\det(E)} \begin{bmatrix} e_{22} & -e_{12} \\ -e_{21} & e_{11} \end{bmatrix}$$

Par symétrie de E

$$\det(E) = e_{11}e_{22} - e_{21}^2 \leq \mu_1^2 - \mu_0^2 \leq (\alpha^2 - 1)\mu_0^2$$

Il est alors posé que $\alpha \in (0, 1)$, de telle sorte que $|\det(E)| \geq (1 - \alpha^2)\mu_0^2$. Il est alors possible de borner supérieurement E^{-1} dans le complément de Schur. En effet

$$|E^{-1}| \leq \frac{1}{(1 - \alpha^2)\mu_0^2} \begin{bmatrix} \alpha & 1 \\ 1 & \alpha \end{bmatrix}$$

et donc

$$|a_{ij}| \leq \mu_0 + \frac{1(1 + \alpha)\mu_0^2}{(1 - \alpha^2)\mu_0} = \left(1 + \frac{2}{1 - \alpha}\right) \mu_0$$

Finalement, pour obtenir la valeur de α , le multiplicateur de μ_0 dans la borne du pivot de 2×2 est rendu équivalent à deux fois le multiplicateur de μ_0 dans la borne du pivot de 1×1 , soit

$$\left(1 + \frac{1}{\alpha}\right)^1 = \left(1 + \frac{2}{1 - \alpha}\right)$$

Ce système peut être résolu et mène à deux valeurs de α , parmi lesquelles on choisit la valeur positive, qui est bien $\alpha = (1 + \sqrt{17})/8$.

Il s'agit d'une dérivation assez simple d'une valeur reprise dans chacune des implémentations auxquelles ce projet s'intéresse. Le concept de choix de pivot et de borne sur l'augmentation des éléments est aussi présent dans d'autres types de factorisation et dans des méthodes de base comme l'élimination de Gauss.

2.2.2 Pseudo-code

Pour l'implémentation, on combine le choix de p à la mise en place de base du problème, vue dans la première section du rapport. Le pseudo-code suivant est ce qui a été suivi pour l'implémentation, présentée dans la partie résultats.

Algorithm 1: Factorisation de Bunch-Parlett (pivotage complet)

Result: $PAP^T = LBL^T$ **Input :** A hermitienne indéfinie**Output:** A modifiée, P

```
1  $\alpha = (1 + \sqrt{17})/8$ 
2  $k = 1$ 
3 while  $k \leq n$  do
4   @viewA[k : n, k : n]
5    $\mu_1 = \max_i |a_{ii}| := |a_{rr}|$ 
6    $\mu_0 = \max_{ij} |a_{ij}| := |a_{pq}|$ 
7   if  $\mu \geq \alpha\mu_0$  then
8      $a_{rr}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
9     Permuter les lignes et colonnes  $k$  et  $r$  dans  $A$ 
10    Permuter les entrées  $k$  et  $r$  dans  $P$ 
11    Insérer le bloc  $E$  dans  $A[k, k]$ 
12    Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
13    Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
14     $k += 1$ 
15    Mettre à jour la vue sur  $A$ 
16  else
17     $a_{pq}$  est le pivot  $2 \times 2$ ,  $s = 2$ 
18    Permuter les lignes et colonnes  $k$  et  $p$  dans  $A$ 
19    Permuter les lignes et colonnes  $k + 1$  et  $q$  dans  $A$ 
20    Permuter les entrées  $k$  et  $p$  dans  $P$ 
21    Permuter les entrées  $k + 1$  et  $q$  dans  $P$ 
22    Insérer le bloc  $E$  dans  $A[k : (k + 1), k : (k + 1)]$ 
23    Insérer les colonnes de  $L$  dans  $A[(k + 1) : n, k : (k + 1)]$ 
24    Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
25     $k += 2$ 
26    Mettre à jour la vue sur  $A$ 
27  end
28 end
```

2.3 Bunch-Kaufman

La stratégie de Bunch-Kaufman pour le choix du pivot est aussi appelée *pivotage partiel*. Comme dans la stratégie de Bunch-Parlett, elle se base sur les valeurs maximales des entrées de la matrice sur et hors diagonale. La différence repose dans le fait qu'elle évite le scannage complet de la matrice pour faire son choix, le décortiquant plutôt en 3 sections de recherche beaucoup plus petite. Le choix du pivot est effectué comme suit;

Poser $\alpha = (1 + \sqrt{17})/8$

Poser $\omega_1 = \max_i |a_{i1}|$ pour i sous diagonale

Si $\omega_1 = 0$, passer à la sous-matrice suivante.

Sinon

Si $|a_{11}| \geq \alpha\omega_1$

Utiliser a_{11} comme pivot 1×1 ($s = 1$, aucune permutation)

Sinon

Poser r , l'index de la valeur maximale dans la colonne 1

Poser $\omega_r = \max_i |a_{ir}|$ pour i sous diagonale

Si $|a_{11}|\omega_r \geq \alpha\omega_1^2$

Utiliser a_{11} comme pivot 1×1 ($s = 1$, aucune permutation)

Sinon

Si $|a_{rr}| \geq \alpha\omega_r$

Utiliser a_{rr} comme pivot 1×1 ($s = 1$, les colonnes et rangées 1 et r sont permutées)

Sinon

Utiliser $\begin{bmatrix} a_{11} & a_{r1} \\ a_{r1} & a_{rr} \end{bmatrix}$ comme pivot 2×2 ($s = 2$, les colonnes et rangées 2 et r sont permutées)

On peut visualiser l'emplacement des pivots comme suit;

$$\begin{pmatrix} a_{11} & \cdots & a_{r1}(\omega_1) & \cdots & \cdots & \cdots \\ \vdots & & \vdots & & & \\ a_{r1}(\omega_1) & \cdots & a_{rr} & \cdots & a_{ir}(\omega_r) & \cdots \\ \vdots & & \vdots & & & \\ \vdots & & a_{ir}(\omega_r) & & & \\ \vdots & & \vdots & & & \end{pmatrix}$$

2.3.1 Pseudo-code

Pour l'implémentation, on reprend la majorité de ce qui a été fait pour Bunch-Parlett, mais en ajustant le choix de p . Le pseudo-code suivant est ce qui a été suivi pour l'implémentation.

Algorithm 2: Factorisation de Bunch-Kaufman (pivotage partiel)

Result: $PAP^T = LBL^T$ **Input :** A hermitienne indéfinie**Output:** A modifiée, P

```
1  $\alpha = (1 + \sqrt{17})/8$ 
2  $k = 1$ 
3 while  $k < n - 1$  do
4   @viewA[ $k : n, k : n$ ]
5    $\omega_1 = \max_i |a_{i1}|$  pour  $i$  sous la diagonale
6   if  $\omega_1 = 0$  then
7     Passer à la sous-matrice suivante
8   else
9     if  $|a_{11}| \geq \alpha\omega_1$  then
10       $a_{11}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
11      Insérer le bloc  $E$  dans  $A[k, k]$ 
12      Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
13      Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
14       $k += 1$ 
15      Mettre à jour la vue sur  $A$ 
16    else
17       $r = \text{index}(\max_i |a_{i1}|)$ 
18       $\omega_r = \max_i |a_{ir}|$  pour  $i$  sous la diagonale
19      if  $|a_{11}|\omega_r \geq \alpha\omega_1^2$  then
20         $a_{11}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
21        Insérer le bloc  $E$  dans  $A[k, k]$ 
22        Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
23        Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
24         $k += 1$ 
25        Mettre à jour la vue sur  $A$ 
26      else
27        if  $|a_{rr}| \geq \alpha\omega_r$  then
28           $a_{rr}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
29          Permuter les lignes et colonnes  $k$  et  $r$  dans  $A$ 
30          Permuter les entrées  $k$  et  $r$  dans  $P$ 
31          Insérer le bloc  $E$  dans  $A[k, k]$ 
32          Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
33          Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
34           $k += 1$ 
35          Mettre à jour la vuesur  $A$ 
36        else
37           $a_{r1}$  est le pivot  $2 \times 2$ ,  $s = 2$ 
38          Permuter les lignes et colonnes  $k + 1$  et  $r$  dans  $A$ 
39          Permuter les entrées  $k + 1$  et  $r$  dans  $P$ 
40          Insérer le bloc  $E$  dans  $A[k : (k + 1), k : (k + 1)]$ 
41          Insérer les colonnes de  $L$  dans  $A[(k + 1) : n, k : (k + 1)]$ 
42          Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
43           $k += 2$ 
44          Mettre à jour la vue sur  $A$ 
45        end
46      end
47    end
48  end
49 end
```

2.4 Rook pivoting

La stratégie de *Rook pivoting* est utilisée lorsque l'on veut éviter que la norme de L soit trop grande (ce qui peut générer des instabilités dans certains algorithmes.) Plusieurs éléments du processus sont les mêmes que pour Bunch-Kaufman, mais le processus comporte une portion itérative. Le choix du pivot est effectué comme suit;

Poser $\alpha = (1 + \sqrt{17})/8$

Poser $\omega_1 = \max_i |a_{i1}|$ pour i sous diagonale

Si $\omega_1 = 0$, passer à la sous-matrice suivante.

Sinon

Si $|a_{11}| \geq \alpha\omega_1$

Utiliser a_{11} comme pivot 1×1 ($s = 1$, aucune permutation)

Sinon

$i = 1$

Répéter jusqu'au choix d'un pivot

Poser r , l'index de la valeur maximale dans la colonne i

Poser $\omega_r = \max_i |a_{ir}|$ pour i sous diagonale

Si $|a_{rr}| \geq \alpha\omega_r$

Utiliser a_{rr} comme pivot 1×1 ($s = 1$, les colonnes et rangées 1 et r sont permutées)

Sinon

Si $\omega_i = \omega_r$

Utiliser $\begin{bmatrix} a_{ii} & a_{ri} \\ a_{ri} & a_{rr} \end{bmatrix}$ comme pivot 2×2 ($s = 2$, les colonnes et rangées 1 et i sont permutées, suivies des colonnes et rangées 2 et r)

Sinon

$i = r$, $\omega_i = \omega_r$

Si un pivot 1×1 est trouvé, le pivot est choisi directement. Sinon, le processus cherche un pivot a_{ri} tel que $|a_{ri}|$ est le maximum dans la rangée r et le maximum dans la colonne i .

2.4.1 Pseudo-code

Le pseudo-code est encore une fois semblable à celui des processus précédent, à la différence de la boucle **while** utilisée dans le choix du pivot.

Algorithm 3: Factorisation avec rook-pivoting

Result: $PAP^T = LBL^T$ **Input :** A hermitienne indéfinie**Output:** A modifiée, P

```
1  $\alpha = (1 + \sqrt{17})/8$ 
2  $k = 1$ 
3 while  $k < n - 1$  do
4    $@view A[k : n, k : n]$ 
5    $\omega_1 = \max_i |a_{i1}|$  pour  $i$  sous la diagonale
6   if  $\omega_1 = 0$  then
7     Passer à la sous-matrice suivante
8   else
9     if  $|a_{11}| \geq \alpha \omega_1$  then
10       $a_{11}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
11      Insérer le bloc  $E$  dans  $A[k, k]$ 
12      Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
13      Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
14       $k += 1$ 
15      Mettre à jour la vue sur  $A$ 
16    else
17       $j = 1$ 
18      while Un pivot n'a pas été choisi do
19         $r = \text{index}(\max_i |a_{ij}|)$ 
20         $\omega_r = \max_i |a_{ir}|$  pour  $i$  sous la diagonale
21        if  $|a_{rr}| \geq \alpha \omega_r$  then
22           $a_{rr}$  est le pivot  $1 \times 1$ ,  $s = 1$ 
23          Permuter les lignes et colonnes  $k$  et  $r$  dans  $A$ 
24          Permuter les entrées  $k$  et  $r$  dans  $P$ 
25          Insérer le bloc  $E$  dans  $A[k, k]$ 
26          Insérer la colonne  $L$  dans  $A[(k + 1) : n, k]$ 
27          Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
28           $k += 1$ 
29          Mettre à jour la vuesur  $A$ 
30        else
31          if  $\omega_i = \omega_r$  then
32             $a_{rj}$  est le pivot  $2 \times 2$ ,  $s = 2$ 
33            Permuter les lignes et colonnes 1 et  $j$  dans  $A$ 
34            Permuter les lignes et colonnes 2 et  $r$  dans  $A$ 
35            Permuter les entrées 1 et  $j$  dans  $P$ 
36            Permuter les entrées 2 et  $r$  dans  $P$ 
37            Insérer le bloc  $E$  dans  $A[k : (k + 1), k : (k + 1)]$ 
38            Insérer les colonnes de  $L$  dans  $A[(k + 1) : n, k : (k + 1)]$ 
39            Mettre à jour le bloc inférieur droit de  $A$  selon le complément de Schur
40             $k += 2$ 
41            Mettre à jour la vue sur  $A$ 
42          else
43             $j = r$ 
44             $\omega_j = \omega_r$ 
45          end
46        end
47      end
48    end
49  end
50 end
```

3 Implémentation

3.1 Stratégies d'optimisation

Différentes stratégies ont été adoptées pour optimiser les performances des fonctions créées, autant d'un point de vue mémoire que vitesse d'exécution.

Tout d'abord, pour se conformer aux normes présentent dans Julia, deux versions de chaque processus a été implémenté, soit une version renvoyant les matrices L , B et P au sein d'une **struct** et une autre modifiant la matrice en place A - le nom de cette dernière contient ainsi un **!** pour signifier la modification des éléments d'entrée.

Pour limiter le nombre d'allocations dans la seconde version, les matrices B et L sont stockées directement aux emplacements mémoire de A tout au long de la factorisation. Dans le schéma suivant, les blocs bleus représentent les blocs de la matrice B et les verts, les colonnes de la matrices L . Toutes les valeurs au dessus de la diagonale de bloc sont mises à 0.

$$A = \begin{pmatrix} \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \end{pmatrix} \rightarrow \begin{pmatrix} \square & 0 & 0 & 0 & 0 & 0 & 0 \\ \square & \square & 0 & 0 & 0 & 0 & 0 \\ \square & \square & \square & 0 & 0 & 0 & 0 \\ \square & \square & \square & \square & 0 & 0 & 0 \\ \square & \square & \square & \square & \square & 0 & 0 \\ \square & \square & \square & \square & \square & \square & 0 \\ \square & \square & \square & \square & \square & \square & \square \end{pmatrix}$$

Pour être en mesure d'extraire les matrices B et L de cette matrice A finale, la position des blocs de taille 2×2 est stockée dans un vecteur. Une fonction a été créer pour permettre à l'utilisateur d'extraire B et L à partir de ce vecteur et de la matrice A , **extract_L_and_B_from_LBL_inplace**.

Dans les deux versions, la symétrie de la matrice est également exploitée pour diminuer le temps d'exécution. En effet, lors de la recherche des valeurs maximales hors diagonales (pour Bunch-Parlett), seule la moitié de la matrice a besoin d'être parcourue. La matrice d'entrée est également établie comme étant Triangulaire inférieure, ce qui permet de limiter l'espace mémoire. Les permutations effectuées sont stockées dans un vecteur P plutôt que dans une matrice de permutation complète.

Les **views** sont utilisées pour limiter le nombre d'allocations et des boucles sont aussi utilisées à plusieurs endroits pour éviter le stockage de vecteurs complets. Pour les pivots de taille 2×2 , le complément de Schur est calculé via son expression algébrique pour encore une fois éviter une allocation de vecteur. Il en va de même pour la mise à jour de L dans le cas 2×2 .

? Autres ajouts Téo ?

Finalement, il est à noter que la notation **i_diagonal** a été préférée à k dans le code pour éviter toute confusion avec la troisième dimension typique en physique ou avec un indice de boucle. Le plein potentiel de la notation Latex (α , μ , etc.) n'a pas non plus exploitée, puisque ces symboles n'apparaissent pas lors de la production des rapports Quarto.

3.2 Bunch-Parlett

Les fonctions suivantes implémentent la factorisation de Bunch-Parlett (pivotage complet) selon les deux approches mentionnées précédemment.

```
function bunch_parlett(A::LowerTriangular)
    """
    Factorise A = LBL' selon la factorisation de Bunch-Parlett, utilisant la technique du pivotage complet
    Entrée : Matrice A carrée, hermitienne et indéfinie
    Sortie : Matrice L triangulaire inférieure
            Matrice B tridiagonale hermitienne
```

```

        Vecteur de permutation P
"""
n = size(A, 1)
T = eltype(A)
subdiagonal = zeros(T, n-1)
diagonal = zeros(T, n)
L = LowerTriangular(Matrix{T}(I, n, n))
P = collect(1:n)

alpha = (1 + sqrt(17))/8
i_diagonal = 1
while i_diagonal < n - 1
    ### Initialisation de la partie de A traitée
    A_ = view(A, i_diagonal:n, i_diagonal:n)
    n_ = size(A_, 1)

    ### Choix du pivot et pivotage
    mu1, r = findmax(abs.(diag(A_)))

    p, q = i_diagonal, i_diagonal
    mu0 = zero(Float64)
    for i in i_diagonal:n
        for j in (i+1):n
            if abs(A[i, j]) > mu0
                = abs(A[i, j])
                p, q = i, j
            end
        end
    end

    if mu1 < alpha * mu0 || i_diagonal == n # E de taille 1, permute les lignes r et 1

        # Permutation dans A_
        perm_r1_et_r2!(A_, 1, r)

        # Permutation dans L
        for j in 1:i_diagonal-1
            L[i_diagonal, j], L[i_diagonal+r-1, j] = L[i_diagonal+r-1, j], L[i_diagonal, j]
        end

        # Permutation dans P
        P[i_diagonal], P[i_diagonal+r-1] = P[i_diagonal+r-1], P[i_diagonal]

        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/mu1^2

        # Complément de Schur
        for i in 2:n_
            a_i1 = A_[i, 1]
            for j in 2:i
                a_j1_conj = conj(A_[j, 1])
                A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
            end
        end
    end
end

```

```

end

# Calcul de B
diagonal[i_diagonal] = A_[1, 1]
subdiagonal[i_diagonal] = 0

# Calcul de L
for i in 2:n_
    L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
end

# Prochain bloc
i_diagonal += 1

else    # E de taille 2, P permute 1 et p et 2 et q

    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, p)
    perm_r1_et_r2!(A_, 2, q)

    # Permutation dans L
    for j in 1:i_diagonal-1
        L[i_diagonal, j], L[i_diagonal+p-1, j] = L[i_diagonal+p-1, j], L[i_diagonal, j]
    end
    for j in 1:i_diagonal-1
        L[i_diagonal+1, j], L[i_diagonal+q-1, j] = L[i_diagonal+q-1, j], L[i_diagonal+1, j]
    end

    # Permutation dans P
    P[i_diagonal], P[i_diagonal+p-1] = P[i_diagonal+p-1], P[i_diagonal]
    P[i_diagonal+1], P[i_diagonal+q-1] = P[i_diagonal+q-1], P[i_diagonal+1]

    # Déterminant de E
    e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
    e_12 = conj(e_21)
    det_E = (e_11*e_22 - e_21*e_12)
    det_E_conj = conj(det_E)
    magnitude_det_E = abs(det_E)
    inv_det_E = det_E_conj/magnitude_det_E^2

    # Complément de Schur
    for i in 3:n_
        a_i1, a_i2 = A_[i, 1], A_[i, 2]
        for j in 3:i
            a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
            A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 - a_i1*e_12)*a_j2_conj)
        end
    end

    # Calcul de B
    subdiagonal[i_diagonal], subdiagonal[i_diagonal+1] = A_[2, 1], 0
    diagonal[i_diagonal], diagonal[i_diagonal + 1] = A_[1, 1], A_[2, 2]

    # Calcul de L

```

```

        for i in i_diagonal + 2:n
            a_i1, a_i2 = A[i, i_diagonal], A[i, i_diagonal+1]
            L[i, i_diagonal] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
            L[i, i_diagonal+1] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
        end

        # Prochain bloc
        i_diagonal += 2
    end

end

if i_diagonal == n - 1
    subdiagonal[i_diagonal] = A[i_diagonal+1, i_diagonal]
    diagonal[i_diagonal], diagonal[i_diagonal+1] = A[i_diagonal, i_diagonal], A[i_diagonal+1, i_diagonal]
else
    diagonal[i_diagonal] = A[i_diagonal, i_diagonal]
end

B = Hermitian(Tridiagonal(subdiagonal, diagonal, conj(subdiagonal)))
result = LBL(L, B, P)

return result
end

function bunch_parlett!(A::LowerTriangular)
    """
    Factorise A = LBL' selon la factorisation de Bunch-Parlett, utilisant la technique du pivotage complexe
    Entrée : Matrice A carrée, hermitienne et indéfinie
    Sortie : Matrice A carrée et hermitienne contenant à la fois L et B :
        - L sur les éléments hors diagonaux
        - B sur les éléments blocs diagonaux ne faisant pas partie de L
    Vecteur de permutation P
    Vecteur de position des blocs 2x2
    """
    n = size(A, 1)
    P = collect(1:n)
    B2x2 = Int[]

    alpha = (1 + sqrt(17))/8
    i_diagonal = 1
    while i_diagonal < n - 1
        ### Initialisation de la partie de A traitée
        A_ = view(A, i_diagonal:n, i_diagonal:n)
        n_ = size(A_, 1)

        ### Choix du pivot et pivotage
        mu1, r = findmax(abs.(diag(A_)))

        p, q = i_diagonal, i_diagonal
        mu0 = zero(Float64)
        for i in i_diagonal:n
            for j in (i+1):n
                if abs(A[i, j]) > mu0

```

```

        = abs(A[i, j])
        p, q = i, j
    end
end
end

if mu1 == alpha * mu0 || i_diagonal == n # E de taille 1, permute les lignes 1 et r
    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, r)

    # Permutation dans L
    for j in 1:i_diagonal-1
        A[i_diagonal, j], A[i_diagonal+r-1, j] = A[i_diagonal+r-1, j], A[i_diagonal, j]
    end

    # Permutation dans P
    P[i_diagonal], P[i_diagonal+r-1] = P[i_diagonal+r-1], P[i_diagonal]

    # Déterminant de E
    a_11_conj = conj(A_[1, 1])
    inv_det_E = a_11_conj/mu1^2

    # Complément de Schur
    for i in 2:n_
        a_i1 = A_[i, 1]
        for j in 2:i
            a_j1_conj = conj(A_[j, 1])
            A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
        end
    end

    # Calcul de L
    for i in 2:n_
        A_[i, 1] *= inv_det_E        # À VALIDER
    end

    # Prochain bloc
    i_diagonal += 1
else
    push!(B2x2, i_diagonal)

    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, p)
    perm_r1_et_r2!(A_, 2, q)

    # Permutation dans L
    for j in 1:i_diagonal-1
        A[i_diagonal, j], A[i_diagonal+p-1, j] = A[i_diagonal+p-1, j], A[i_diagonal, j]
    end
    for j in 1:i_diagonal-1
        A[i_diagonal+1, j], A[i_diagonal+q-1, j] = A[i_diagonal+q-1, j], A[i_diagonal+1, j]
    end
end

```

```

# Permutation dans P
P[i_diagonal], P[i_diagonal+p-1] = P[i_diagonal+p-1], P[i_diagonal]
P[i_diagonal+1], P[i_diagonal+q-1] = P[i_diagonal+q-1], P[i_diagonal+1]

# Déterminant de E
e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
e_12 = conj(e_21)
det_E = (e_11*e_22 - e_21*e_12)
det_E_conj = conj(det_E)
magnitude_det_E = abs(det_E)
inv_det_E = det_E_conj/magnitude_det_E^2

# Complément de Schur
for i in 3:n_
    a_i1, a_i2 = A_[i, 1], A_[i, 2]
    for j in 3:i
        a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
        A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 - a_i1*e_12)*a_j2_conj)
    end
end

# Calcul de L
for i in 3:n_
    a_i1, a_i2 = A_[i, 1], A_[i, 2]
    A_[i, 1] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
    A_[i, 2] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
end

# Prochain bloc
i_diagonal += 2
end

end

return P, B2x2
end

```

3.3 Bunch-Kaufman

Les fonctions suivantes implémentent la factorisation de Bunch-Kaufman (pivotage partiel) selon les deux approches mentionnées précédemment.

```

function bunch_kaufman(A::LowerTriangular)
    """
    Factorise A selon la factorisation LBL' de Bunch-Kaufman, qui utilise la technique de pivotage partiel.
    Entrée : - Matrice A carrée, hermitienne et indéfinie
    Sortie : - Structure qui contient :
                - Matrice L triangulaire inférieure
                - Matrice B tridiagonale hermitienne
                - Vecteur de permutation vec_P
    """
    n = size(A, 1)
    T = eltype(A)
    subdiagonal = zeros(T, n-1)

```

```

diagonal = zeros(T, n)
L = LowerTriangular(Matrix{T}(I, n, n))
vec_P = collect(1:n)

alpha = (1 + sqrt(17))/8
i_diagonal = 1
while i_diagonal < n - 1
    ### Initialisation de la partie de A traitée
    A_ = view(A, i_diagonal:n, i_diagonal:n)
    n_ = size(A_, 1)

    ### Choix du pivot et pivotage
    w_1 = -1.0
    r = 1
    for i in 2:n_
        magnitude_a_i1 = abs(A_[i, 1])
        if magnitude_a_i1 > w_1
            w_1 = magnitude_a_i1
            r = i
        end
    end

    magnitude_a_11 = abs(A_[1, 1])
    if magnitude_a_11 >= alpha*w_1 # E de taille 1, P ne permute rien
        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/magnitude_a_11^2

        # Complément de Schur
        for i in 2:n_
            a_i1 = A_[i, 1]
            for j in 2:i
                a_j1_conj = conj(A_[j, 1])
                A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
            end
        end

        # Calcul de B
        diagonal[i_diagonal] = A_[1, 1]
        subdiagonal[i_diagonal] = 0

        # Calcul de L
        for i in 2:n_
            L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
        end

        # Prochain bloc
        i_diagonal += 1
    else
        w_r = -1.0
        for j in 1:r-1
            magnitude_a_ir = abs(A_[r, j])
            if magnitude_a_ir > w_r
                w_r = magnitude_a_ir
            end
        end
    end
end

```



```

end
end
for i in r+1:n_
    magnitude_a_ir = abs(A_[i, r])
    if magnitude_a_ir > w_r
        w_r = magnitude_a_ir
    end
end
end

if magnitude_a_11*w_r >= alpha*w_1^2 # E de taille 1, P ne permute rien
    # Déterminant de E
    a_11_conj = conj(A_[1, 1])
    inv_det_E = a_11_conj/magnitude_a_11^2

    # Complément de Schur
    for i in 2:n_
        a_i1 = A_[i, 1]
        for j in 2:i
            a_j1_conj = conj(A_[j, 1])
            A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
        end
    end

    # Calcul de B
    diagonal[i_diagonal] = A_[1, 1]
    subdiagonal[i_diagonal] = 0

    # Calcul de L
    for i in 2:n_
        L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
    end

    # Prochain bloc
    i_diagonal += 1
else
    magnitude_a_rr = abs(A_[r, r])
    if magnitude_a_rr >= alpha*w_r # E de taille 1, P permute 1 et r
        # Permutation dans A_
        perm_r1_et_r2!(A_, 1, r)

        # Permutation dans L
        for j in 1:i_diagonal-1
            L[i_diagonal, j], L[i_diagonal+r-1, j] = L[i_diagonal+r-1, j], L[i_diagonal, j]
        end

        # Permutation dans P
        vec_P[i_diagonal], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1], vec_P[i_diagonal]

        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/magnitude_a_rr^2

        # Complément de Schur
        for i in 2:n_

```

```

        a_i1 = A_[i, 1]
        for j in 2:i
            a_j1_conj = conj(A_[j, 1])
            A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
        end
    end

    # Calcul de B
    diagonal[i_diagonal] = A_[1, 1]
    subdiagonal[i_diagonal] = 0

    # Calcul de L
    for i in 2:n_
        L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
    end

    # Prochain bloc
    i_diagonal += 1
else # E de taille 2, P permute 2 et r
    # Permutation dans A_
    perm_r1_et_r2!(A_, 2, r)

    # Permutation dans L
    for j in 1:i_diagonal-1
        L[i_diagonal+1, j], L[i_diagonal+r-1, j] = L[i_diagonal+r-1, j], L[i_diagonal+1, j]
    end

    # Permutation dans P
    vec_P[i_diagonal+1], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1], vec_P[i_diagonal+1]

    # Déterminant de E
    e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
    e_12 = conj(e_21)
    det_E = (e_11*e_22 - e_21*e_12)
    det_E_conj = conj(det_E)
    magnitude_det_E = abs(det_E)
    inv_det_E = det_E_conj/magnitude_det_E^2

    # Complément de Schur
    for i in 3:n_
        a_i1, a_i2 = A_[i, 1], A_[i, 2]
        for j in 3:i
            a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
            A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 - a_i1*e_12)*a_j2_conj)
        end
    end

    # Calcul de B
    subdiagonal[i_diagonal], subdiagonal[i_diagonal+1] = A_[2, 1], 0
    diagonal[i_diagonal], diagonal[i_diagonal + 1] = A_[1, 1], A_[2, 2]

    # Calcul de L
    for i in i_diagonal + 2:n
        a_i1, a_i2 = A[i, i_diagonal], A[i, i_diagonal+1]

```

```

        L[i, i_diagonal] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
        L[i, i_diagonal+1] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
    end

    # Prochain bloc
    i_diagonal += 2
end

end

end

end

# Dernier bloc
if i_diagonal == n - 1
    subdiagonal[i_diagonal] = A[i_diagonal+1, i_diagonal]
    diagonal[i_diagonal], diagonal[i_diagonal+1] = A[i_diagonal, i_diagonal], A[i_diagonal+1, i_diagonal]
else
    diagonal[i_diagonal] = A[i_diagonal, i_diagonal]
end

B = Hermitian(Tridiagonal(subdiagonal, diagonal, conj(subdiagonal)))
result = LBL(L, B, vec_P)
return result
end

function bunch_kaufman!(A::LowerTriangular)
    """
    Modifie A sous la forme compacte de la factorisation LBL' de Bunch-Kaufman, qui utilise la technique de
    L et B sont stockés en écrasant A, où L se situe en dessous des éléments blocs diagonaux de B.
    Entrée : - Matrice A carrée, hermitienne et indéfinie
    Sortie : - Vecteur de permutation vec_P
              - Vecteur de position des blocs diagonaux 2x2 vec_2by2
    """
    n = size(A, 1)
    vec_P = collect(1:n)
    vec_2by2 = Int[]

    alpha = (1 + sqrt(17))/8
    i_diagonal = 1
    while i_diagonal < n - 1
        ### Initialisation de la partie de A traitée
        A_ = view(A, i_diagonal:n, i_diagonal:n)
        n_ = size(A_, 1)

        ### Choix du pivot et pivotage
        w_1 = -1.0
        r = 1
        for i in 2:n_
            magnitude_a_i1 = abs(A_[i, 1])
            if magnitude_a_i1 > w_1
                w_1 = magnitude_a_i1
                r = i
            end
        end
        i_diagonal += 2
    end
end

```

```

magnitude_a_11 = abs(A_[1, 1])
if magnitude_a_11 >= alpha*w_1 # E de taille 1, P ne permute rien
    # Déterminant de E
    a_11_conj = conj(A_[1, 1])
    inv_det_E = a_11_conj/magnitude_a_11^2

    # Complément de Schur
    for i in 2:n_
        a_i1 = A_[i, 1]
        for j in 2:i
            a_j1_conj = conj(A_[j, 1])
            A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
        end
    end

    # Calcul de L
    for i in 2:n_
        A_[i, 1] *= inv_det_E
    end

    # Prochain bloc
    i_diagonal += 1
else
    w_r = -1.0
    for j in 1:r-1
        magnitude_a_ir = abs(A_[r, j])
        if magnitude_a_ir > w_r
            w_r = magnitude_a_ir
        end
    end

    for i in r+1:n_
        magnitude_a_ir = abs(A_[i, r])
        if magnitude_a_ir > w_r
            w_r = magnitude_a_ir
        end
    end

    if magnitude_a_11*w_r >= alpha*w_1^2 # E de taille 1, P ne permute rien
        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/magnitude_a_11^2

        # Complément de Schur
        for i in 2:n_
            a_i1 = A_[i, 1]
            for j in 2:i
                a_j1_conj = conj(A_[j, 1])
                A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
            end
        end

        # Calcul de L
        for i in 2:n_
            A_[i, 1] *= inv_det_E
        end
    end
end

```

```

end

# Prochain bloc
i_diagonal += 1
else
  magnitude_a_rr = abs(A_[r, r])
  if magnitude_a_rr >= alpha*w_r # E de taille 1, P permute 1 et r
    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, r)

    # Permutation dans L
    for j in 1:i_diagonal-1
      A[i_diagonal, j], A[i_diagonal+r-1, j] = A[i_diagonal+r-1, j], A[i_diagonal, j]
    end

    # Permutation dans P
    vec_P[i_diagonal], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1], vec_P[i_diagonal]

    # Déterminant de E
    a_11_conj = conj(A_[1, 1])
    inv_det_E = a_11_conj/magnitude_a_rr^2

    # Complément de Schur
    for i in 2:n_
      a_i1 = A_[i, 1]
      for j in 2:i
        a_j1_conj = conj(A_[j, 1])
        A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
      end
    end

    # Calcul de L
    for i in 2:n_
      A_[i, 1] *= inv_det_E
    end

    # Prochain bloc
    i_diagonal += 1
  else # E de taille 2, P permute 2 et r
    push!(vec_2by2, i_diagonal)

    # Permutation dans A_
    perm_r1_et_r2!(A_, 2, r)

    # Permutation dans L
    for j in 1:i_diagonal-1
      A[i_diagonal+1, j], A[i_diagonal+r-1, j] = A[i_diagonal+r-1, j], A[i_diagonal+1, j]
    end

    # Permutation dans P
    vec_P[i_diagonal+1], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1], vec_P[i_diagonal+1]

    # Déterminant de E
    e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]

```

```

        e_12 = conj(e_21)
        det_E = (e_11*e_22 - e_21*e_12)
        det_E_conj = conj(det_E)
        magnitude_det_E = abs(det_E)
        inv_det_E = det_E_conj/magnitude_det_E^2

        # Complément de Schur
        for i in 3:n_
            a_i1, a_i2 = A_[i, 1], A_[i, 2]
            for j in 3:i
                a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
                A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 - a_i1*e_12)*a_j2_conj)
            end
        end

        # Calcul de L
        for i in 3:n_
            a_i1, a_i2 = A_[i, 1], A_[i, 2]
            A_[i, 1] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
            A_[i, 2] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
        end

        # Prochain bloc
        i_diagonal += 2
    end
end
end
end

return vec_P, vec_2by2
end

```

3.4 *Rook pivoting*

Les fonctions suivantes implémentent la factorisation avec *rook pivoting* selon les deux approches mentionnées précédemment.

```

function bunch_kaufman_rook(A::LowerTriangular)
    """
    Factorise A selon la factorisation LBL' de Bunch-Kaufman, qui utilise la technique du 'rook pivoting'.
    Entrée : - Matrice A carrée, hermitienne et indéfinie
    Sortie : - Structure qui contient :
                - Matrice L triangulaire inférieure
                - Matrice B tridiagonale hermitienne
                - Vecteur de permutation vec_P
    """
    n = size(A, 1)
    T = eltype(A)
    subdiagonal = zeros(T, n-1)
    diagonal = zeros(T, n)
    L = LowerTriangular(Matrix{T}(I, n, n))
    vec_P = collect(1:n)

    alpha = (1 + sqrt(17))/8
    i_diagonal = 1

```

```

while i_diagonal < n - 1
    ### Initialisation de la partie de A traitée
    A_ = view(A, i_diagonal:n, i_diagonal:n)
    n_ = size(A_, 1)

    ### Choix du pivot et pivotage
    w_1 = -1.0
    r = 1
    for i in 2:n_
        magnitude_a_i1 = abs(A_[i, 1])
        if magnitude_a_i1 > w_1
            w_1 = magnitude_a_i1
            r = i
        end
    end

    magnitude_a_11 = abs(A_[1, 1])
    if magnitude_a_11 >= alpha*w_1 # E de taille 1, P ne permute rien
        # Déterminant de E
        a_11_conj = conj(A_[1, 1])
        inv_det_E = a_11_conj/magnitude_a_11^2

        # Complément de Schur
        for i in 2:n_
            a_i1 = A_[i, 1]
            for j in 2:i
                a_j1_conj = conj(A_[j, 1])
                A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
            end
        end

        # Calcul de B
        diagonal[i_diagonal] = A_[1, 1]
        subdiagonal[i_diagonal] = 0

        # Calcul de L
        for i in 2:n_
            L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
        end

        # Prochain bloc
        i_diagonal += 1
    else
        index = 1
        w_index = w_1
        r_new = r
        repeat = true
        while repeat
            w_r = -1.0
            # Subdiagonal
            for i in r+1:n_
                magnitude_a_ir = abs(A_[i, r])
                if magnitude_a_ir > w_r
                    w_r = magnitude_a_ir
                end
            end
        end
    end
end

```

```

        r_new = i
    end
end
# Superdiagonal
for j in 1:r-1
    magnitude_a_ir = abs(A_[r, j])
    if magnitude_a_ir > w_r
        w_r = magnitude_a_ir
    end
end
end

magnitude_a_rr = abs(A_[r, r])
if magnitude_a_rr >= alpha*w_r # E de taille 1, P permute 1 et r
    repeat = false

    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, r)

    # Permutation dans L
    for j in 1:i_diagonal-1
        L[i_diagonal, j], L[i_diagonal+r-1, j] = L[i_diagonal+r-1, j], L[i_diagonal, j]
    end

    # Permutation dans P
    vec_P[i_diagonal], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1], vec_P[i_diagonal]

    # Déterminant de E
    a_11_conj = conj(A_[1, 1])
    inv_det_E = a_11_conj/magnitude_a_rr^2

    # Complément de Schur
    for i in 2:n_
        a_i1 = A_[i, 1]
        for j in 2:i
            a_j1_conj = conj(A_[j, 1])
            A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
        end
    end

    # Calcul de B
    diagonal[i_diagonal] = A_[1, 1]
    subdiagonal[i_diagonal] = 0

    # Calcul de L
    for i in 2:n_
        L[i_diagonal+i-1, i_diagonal] = inv_det_E*A_[i, 1]
    end

    # Prochain bloc
    i_diagonal += 1
elseif w_index == w_r # E de taille 2, P permute 1 et index, puis 2 et r
    repeat = false

    # Permutation dans A_

```



```

perm_r1_et_r2!(A_, 1, index)
perm_r1_et_r2!(A_, 2, r)

# Permutation dans L
for j in 1:i_diagonal-1
    L[i_diagonal, j], L[i_diagonal+index-1, j] = L[i_diagonal+index-1, j], L[i_diagonal, j]
end
for j in 1:i_diagonal-1
    L[i_diagonal+1, j], L[i_diagonal+r-1, j] = L[i_diagonal+r-1, j], L[i_diagonal+1, j]
end

# Permutation dans P
vec_P[i_diagonal], vec_P[i_diagonal+index-1] = vec_P[i_diagonal+index-1], vec_P[i_diagonal]
vec_P[i_diagonal+1], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1], vec_P[i_diagonal+1]

# Déterminant de E
e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
e_12 = conj(e_21)
det_E = (e_11*e_22 - e_21*e_12)
det_E_conj = conj(det_E)
magnitude_det_E = abs(det_E)
inv_det_E = det_E_conj/magnitude_det_E^2

# Complément de Schur
for i in 3:n_
    a_i1, a_i2 = A_[i, 1], A_[i, 2]
    for j in 3:i
        a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
        A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 - a_i1*e_12))
    end
end

# Calcul de B
subdiagonal[i_diagonal], subdiagonal[i_diagonal+1] = A_[2, 1], 0
diagonal[i_diagonal], diagonal[i_diagonal + 1] = A_[1, 1], A_[2, 2]

# Calcul de L
for i in i_diagonal + 2:n
    a_i1, a_i2 = A[i, i_diagonal], A[i, i_diagonal+1]
    L[i, i_diagonal] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
    L[i, i_diagonal+1] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
end

# Prochain bloc
i_diagonal += 2
else
    index = r
    w_index = w_r
    r = r_new
end
end
end
end
end

```

```

# Dernier bloc
if i_diagonal == n - 1
    subdiagonal[i_diagonal] = A[i_diagonal+1, i_diagonal]
    diagonal[i_diagonal], diagonal[i_diagonal+1] = A[i_diagonal, i_diagonal], A[i_diagonal+1, i_diagonal]
else
    diagonal[i_diagonal] = A[i_diagonal, i_diagonal]
end

B = Hermitian(Tridiagonal(subdiagonal, diagonal, conj(subdiagonal)))
result = LBL(L, B, vec_P)
return result
end

function bunch_kaufman_rook!(A::LowerTriangular)
    """
    Modifie A sous la forme compacte de la factorisation LBL' de Bunch-Kaufman, qui utilise la technique de
    L et B sont stockés en écrasant A, où L se situe en dessous des éléments blocs diagonaux de B.
    Entrée : - Matrice A carrée, hermitienne et indéfinie
    Sortie : - Vecteur de permutation vec_P
              - Vecteur de position des blocs diagonaux 2x2 vec_2by2
    """
    n = size(A, 1)
    vec_P = collect(1:n)
    vec_2by2 = Int[]

    alpha = (1 + sqrt(17))/8
    i_diagonal = 1
    while i_diagonal < n - 1
        ### Initialisation de la partie de A traitée
        A_ = view(A, i_diagonal:n, i_diagonal:n)
        n_ = size(A_, 1)

        ### Choix du pivot et pivotage
        w_1 = -1.0
        r = 1
        for i in 2:n_
            magnitude_a_i1 = abs(A_[i, 1])
            if magnitude_a_i1 > w_1
                w_1 = magnitude_a_i1
                r = i
            end
        end

        magnitude_a_11 = abs(A_[1, 1])
        if magnitude_a_11 >= alpha*w_1 # E de taille 1, P ne permute rien
            # Déterminant de E
            a_11_conj = conj(A_[1, 1])
            inv_det_E = a_11_conj/magnitude_a_11^2

            # Complément de Schur
            for i in 2:n_
                a_i1 = A_[i, 1]
                for j in 2:i
                    a_j1_conj = conj(A_[j, 1])

```

```

        A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
    end
end

# Calcul de L
for i in 2:n_
    A_[i, 1] *= inv_det_E
end

# Prochain bloc
i_diagonal += 1
else
    index = 1
    w_index = w_1
    r_new = r
    repeat = true
    while repeat
        w_r = -1.0
        # Subdiagonal
        for i in r+1:n_
            magnitude_a_ir = abs(A_[i, r])
            if magnitude_a_ir > w_r
                w_r = magnitude_a_ir
                r_new = i
            end
        end
        # Superdiagonal
        for j in 1:r-1
            magnitude_a_ir = abs(A_[r, j])
            if magnitude_a_ir > w_r
                w_r = magnitude_a_ir
            end
        end
        magnitude_a_rr = abs(A_[r, r])
        if magnitude_a_rr >= alpha*w_r # E de taille 1, P permute 1 et r
            repeat = false

            # Permutation dans A_
            perm_r1_et_r2!(A_, 1, r)

            # Permutation dans L
            for j in 1:i_diagonal-1
                A[i_diagonal, j], A[i_diagonal+r-1, j] = A[i_diagonal+r-1, j], A[i_diagonal, j]
            end

            # Permutation dans P
            vec_P[i_diagonal], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1], vec_P[i_diagonal]

            # Déterminant de E
            a_11_conj = conj(A_[1, 1])
            inv_det_E = a_11_conj/magnitude_a_rr^2

            # Complément de Schur

```

```

for i in 2:n_
    a_i1 = A_[i, 1]
    for j in 2:i
        a_j1_conj = conj(A_[j, 1])
        A_[i, j] -= inv_det_E * a_i1 * a_j1_conj
    end
end

# Calcul de L
for i in 2:n_
    A_[i, 1] *= inv_det_E
end

# Prochain bloc
i_diagonal += 1
elseif w_index == w_r # E de taille 2, P permute 1 et index, puis 2 et r
    repeat = false

    push!(vec_2by2, i_diagonal)

    # Permutation dans A_
    perm_r1_et_r2!(A_, 1, index)
    perm_r1_et_r2!(A_, 2, r)

    # Permutation dans L
    for j in 1:i_diagonal-1
        A[i_diagonal, j], A[i_diagonal+index-1, j] = A[i_diagonal+index-1, j], A[i_diagonal, j]
    end
    for j in 1:i_diagonal-1
        A[i_diagonal+1, j], A[i_diagonal+r-1, j] = A[i_diagonal+r-1, j], A[i_diagonal+1, j]
    end

    # Permutation dans P
    vec_P[i_diagonal], vec_P[i_diagonal+index-1] = vec_P[i_diagonal+index-1], vec_P[i_diagonal]
    vec_P[i_diagonal+1], vec_P[i_diagonal+r-1] = vec_P[i_diagonal+r-1], vec_P[i_diagonal+1]

    # Déterminant de E
    e_11, e_22, e_21 = A_[1, 1], A_[2, 2], A_[2, 1]
    e_12 = conj(e_21)
    det_E = (e_11*e_22 - e_21*e_12)
    det_E_conj = conj(det_E)
    magnitude_det_E = abs(det_E)
    inv_det_E = det_E_conj/magnitude_det_E^2

    # Complément de Schur
    for i in 3:n_
        a_i1, a_i2 = A_[i, 1], A_[i, 2]
        for j in 3:i
            a_j1_conj, a_j2_conj = conj(A_[j, 1]), conj(A_[j, 2])
            A_[i, j] -= inv_det_E*((a_i1*e_22 - a_i2*e_21)*a_j1_conj + (a_i2*e_11 - a_i1*e_12)*a_j2_conj)
        end
    end

    # Calcul de L

```

```

        for i in 3:n_
            a_i1, a_i2 = A_[i, 1], A_[i, 2]
            A_[i, 1] = inv_det_E*(a_i1*e_22 - a_i2*e_21)
            A_[i, 2] = inv_det_E*(a_i2*e_11 - a_i1*e_12)
        end

        # Prochain bloc
        i_diagonal += 2
    else
        index = r
        w_index = w_r
        r = r_new
    end
end
end
end

return vec_P, vec_2by2
end

```

4 Tests et performances

Trois types de tests ont été effectués sur les implémentations. Les premières séries de tests visent à valider que les factorisations fonctionnent sur un ensemble de type de données, soit des **Float64**, des **ComplexF64** et finalement des **BigFloat**. La seconde série de tests vise à évaluer et comparer les performances des différentes implémentations. Finalement, la dernière série vise à utiliser la factorisation pour la résolution de différents systèmes de point de selle.

Dans le fichier **Code**, la section *Utilitaire* liste plusieurs des fonctions nécessaires au roulement de ces tests, en particulier les tests sur le type de données. Les tests sont ensuite présentés dans l'ordre mentionnés, suite aux implémentations.

4.1 Type de données

Pour tester les fonctions sur différents types de données, 25 matrices de forme

$$K = \begin{bmatrix} I & A \\ A^* & 0 \end{bmatrix}$$

sont créées l'une après l'autre, où A est aléatoire et de taille 50×25 . Les matrices sont factorisées et le résultat de la factorisation $P^T L B L^T P$ est comparé à A via la norme de leur différence. La tolérance des tests est de $1E - 10$. Il est aussi validé que le type des données contenues dans L et B est le même que celui des éléments de A . Toutes les implémentations passent les tests en ce qui concerne le type de données supportées. (à vérifier, rook)

4.2 Benchmarking

La même approche que celle décrite à la section précédente est utilisée pour effectuer le **Benchmarking** des différentes implémentations, soit une matrice K de taille 75×75 construite à partir d'une matrice A aléatoire. La factorisation est aussi effectuée en utilisant la fonction **bunchkaufman** de Julia, qui provient de LAPACK. L'option **rook** de cette dernière devait être testée, mais elle ne semble pas fonctionner - la documentation en ligne est peut-être erronée. Les résultats obtenus sont présentés dans la cellule ci-dessous et les résultats commentés dans la section Discussion.

```

Performance de bunch_parlett:
- Allocations : 816
- Temps d'exécution : 1.1071260762331838e7
Performance de bunch_parlett!:
- Allocations : 807
- Temps d'exécution : 1.052149978858351e7
Performance de bunch_kaufman:
- Allocations : 18
- Temps d'exécution : 9.03371e6
Performance de bunch_kaufman!:
- Allocations : 10
- Temps d'exécution : 9.7168259765625e6
Performance de bunch_kaufman_rook:
- Allocations : 22
- Temps d'exécution : 9.522212092130518e6
Performance de bunch_kaufman_rook!:
- Allocations : 14
- Temps d'exécution : 9.978934607645875e6
Performance de bunchkaufman LAPACK:
- Allocations : 73
- Temps d'exécution : 6.522576152832675e6
Performance de bunchkaufman! LAPACK:
- Allocations : 67
- Temps d'exécution : 6.170483395755306e6

```

4.3 Résolution de systèmes aux moindres carrés

Pour la résolution de systèmes, deux types de tests sont effectués. D'abord, des systèmes symétriques indéfinis aléatoires construits de la même manière que pour les deux autres séries de tests. Les vecteurs b sont définis de manière aléatoire et les solutions obtenues sont comparées aux solutions obtenues par factorisation `qr`. La définition de la fonction `\` a été étendue pour spécifiquement s'appliquer au format des problèmes qui nous intéressent.

```

import Base: \

function \{F::LBL{T}, b::AbstractVector{T}) where T
    L, B, vec_P = F.L, F.B, F.vec_P
    n = size(vec_P, 1)

    # Permuter b : b_perm = P * b
    b_perm = b[vec_P]

    # Résoudre L * y = b_perm <=> y = L \ b_perm
    y = similar(b)
    for i in 1:n
        y[i] = b_perm[i]
        for j in 1:i-1
            y[i] -= L[i, j]*y[j]
        end
    end

    # Résoudre B * z = y <=> z = B \ y
    z = similar(b)
    i = 1
    while i < n-1

```

```

        if B[i+1, i] == 0
            z[i] = y[i]/B[i, i]

            i += 1
        else
            B_11, B_22, B_21 = B[i, i], B[i+1, i+1], B[i+1, i]
            B_12 = conj(B_21)
            det = (B_11*B_22 - B_21*B_12)
            inv_det = conj(det)/abs(det)^2
            z[i] = inv_det*(B_22*y[i] - B_12*y[i+1])
            z[i+1] = inv_det*(B_11*y[i+1] - B_21*y[i])

            i += 2
        end
    end
end
if B[n, n-1] == 0
    z[n-1] = y[n-1]/B[n-1, n-1]
    z[n] = y[n]/B[n, n]
else
    B_11, B_22, B_21 = B[n-1, n-1], B[n, n], B[n, n-1]
    B_12 = conj(B_21)
    det = (B_11*B_22 - B_21*B_12)
    inv_det = conj(det)/abs(det)^2
    z[n-1] = inv_det*(B_22*y[n-1] - B_12*y[n])
    z[n] = inv_det*(B_11*y[n] - B_21*y[n-1])
end

# Résoudre L'* x_perm = z <=> x_perm = L' \ z
x_perm = similar(b)
for i in n:-1:1
    x_perm[i] = z[i]
    for j in n:-1:i+1
        x_perm[i] -= conj(L[j, i])*x_perm[j]
    end
end

# Permuter x_perm : x = P' * x_perm
x = similar(b)
x[vec_P] = x_perm # Inverse de P est P' car P est une permutation

return x
end

```

Les tests sont effectués sur 100 problèmes aléatoires différents et l'erreur moyenne sur x est présentée ci-dessous.

Average error on solution:

```

- bunch_parlett : 2.1726034227706443e-72
- bunch_parlett! : 2.1726034227706443e-72
- bunch_kaufman : 1.5621733916746804e-76
- bunch_kaufman! : 1.5621733916746804e-76
- bunch_kaufman_rook : 4.789401571869194e-74
- bunch_kaufman_rook! : 4.789401571869194e-74
- bunch_kaufman de LAPACK : 2.841496814172852e-76
- bunch_kaufman! de LAPACK : 2.841496814172852e-76

```

Ensuite, quatre problèmes issus de **MatrixMarket** sont résolus avec les différentes approches, soient **ch5-5-b1**, **ch4-4-b1**, **ch4-4-b2** et **n3c5-b2**. Les systèmes de point de selle sont générées comme dans les sections précédentes, mais en incluant cette fois une légère perturbation dans le bloc inférieur droit des matrices K . Les résultats obtenus sont présentés ci-dessous.

```
Error on residual for ch5-5-b1 :
- bunch_parlett : 0.0
- bunch_parlett! : 0.0
- bunch_kaufman : 6.3377057450964e-6
- bunch_kaufman! : 6.3377057450964e-6
- bunch_kaufman_rook : 6.3377057450964e-6
- bunch_kaufman_rook! : 6.3377057450964e-6
Error on residual for ch4-4-b1 :
- bunch_parlett : 0.0
- bunch_parlett! : 0.0
- bunch_kaufman : 0.00047481804281535034
- bunch_kaufman! : 0.00047481804281535034
- bunch_kaufman_rook : 0.00047481804281535034
- bunch_kaufman_rook! : 0.00047481804281535034
Error on residual for ch4-4-b2 :
- bunch_parlett : 0.0
- bunch_parlett! : 0.0
- bunch_kaufman : 8.881784197001252e-16
- bunch_kaufman! : 8.881784197001252e-16
- bunch_kaufman_rook : 0.0
- bunch_kaufman_rook! : 0.0
Error on residual for n3c5-b2 :
- bunch_parlett : 0.0
- bunch_parlett! : 0.0
- bunch_kaufman : 0.0
- bunch_kaufman! : 0.0
- bunch_kaufman_rook : 0.0
- bunch_kaufman_rook! : 0.0
Average error on residual:
- bunch_parlett : 0.0
- bunch_parlett! : 0.0
- bunch_kaufman : 0.00012028893714033373
- bunch_kaufman! : 0.00012028893714033373
- bunch_kaufman_rook : 0.00012028893714011168
- bunch_kaufman_rook! : 0.00012028893714011168
```

5 Discussion

5.1 Comparaison des processus

D'un point de vue théorique, un théorème permet d'assurer la stabilité du processus de Bunch-Kaufman. Ce théorème assure que pour $(A + \Delta A)\hat{x} = b$ la solution du système $\hat{x}$, la perturbation sur la norme de A est bornée supérieurement comme $\|\Delta A\|_M \leq p(n)\rho_n u \|A\|_M + O(u^2)$ où p est quadratique. Ce résultat s'applique si le système est résolu par élimination gaussienne ou en utilisant l'inverse. Aucun résultat semblable n'est associé à Bunch-Parlett.

Pour tous les processus d'intérêt dans ce projet, un autre résultat intéressant est le suivant:

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq p(n)u \text{cond}(A) \text{cond}(|D||L^T|) + O(u^2)$$

$\text{cond}(|D||L^T|)$ n'est pas borné pour le pivotage partiel, mais il l'est pour le *rook pivoting* et cette méthode est donc en théorie moins sensible à un A initial moins bien conditionné.

Du point de vue de nos résultats, le **Benchmarking** indique que nos implémentations effectue la factorisation avec moins d'allocations que la version de LAPACK, les rendant ainsi pratiquement aussi rapide que ces dernières pour des matrices de petites tailles. Le nombre d'allocation est semblable pour les 3 stratégies de pivotage, avec les version ! en nécessitant évidemment moins dans tous les cas.

Du point de vue du temps d'exécutions, il était attendu que Bunch-Parlett soit le processus demandant le plus grand nombre de *flop* et c'est bien la méthode avec le plus long temps d'exécution.

Dans la résolution de problèmes aux moindre carrés, notre implémentation de Bunch-Kaufman affiche une erreur inférieure à celle obtenue avec l'implémentation de LAPACK. Bunch-Parlett présente la plus grande erreur, mais la différence demeure minime en **Big Float**. Les versions renvoyant les matrices L et B directement et celle stockant les valeurs dans A donnent la même erreur. C'est aussi le cas de l'implémentation en LAPACK, ce qui nous indique que c'est bien le comportement attendu.

Pour les problèmes issus de **MatrixMarket**, il y a une plus grande différences dans les résultats. Pour l'ensemble des problèmes, Bunch-Parlett présente le résidu le plus faible, inférieur à l'épsilon machine. Pour les deux premiers problèmes, les implémentations de Bunch-Kaufman et de *rook pivoting* présentent les mêmes erreurs assez élevées. Il est difficile d'identifier pourquoi; les deux premiers problèmes sont ceux créés à partir des matrices au nombre de conditionnement de plus bas, mais aux normes les plus élevées. Le noyau des matrices A des deux problèmes est aussi de taille 1, ce qui n'est pas le cas des deux derniers problèmes, pour lesquels la dimension du noyau est plus élevée.

5.2 Difficultés rencontrées

6 Retour sur les objectifs

La majorité des objectifs du projet sont accomplis avec la remise de la phase 2.

1. Comprendre et documenter les factorisations de Bunch-Parlett et Bunch-Kaufman (pivotage complet et pivotage partiel.) Fournir des exemples, des schémas explicatifs, faire des liens avec les éléments vus en classe.
 - Voir la section Section 1 et la section Section 2.
2. Implémenter ces deux factorisation ainsi que l'alternative du *rook pivoting*. Les comparer et identifier leurs avantages et leurs inconvénients respectifs.
 - Voir la section Section 3 et la section Section 5.
 - Le nombre de flop lui-même n'a pas été comparé directement, d'un point de vue numérique.
 - La sous-section Section 3.1 explique comment les fonctions ont été optimisées pour réduire le temps d'exécution ainsi que le nombre d'allocation.
3. Comparer nos implémentations à celles déjà disponibles dans Julia et tenter d'identifier les sources possibles des différences observées.
 - Voir la section Section 4.
 - Il a été confirmé qu'aucune implémentation de Bunch-Parlett n'existe dans Julia.
 - L'option *rook pivoting* devait être testée, mais ne fonctionnait pas lors de nos tests.
4. Joindre la factorisation à des méthodes de résolution de systèmes linéaires.
 - Voir la section Section 4, plus spécifiquement la redéfinition de la fonction $\backslash$.
5. Appliquer l'implémentation à un ensemble de problèmes modèles ainsi qu'à un problème plus spécifique au choix.
 - Voir la section Section 4.

- Différents problèmes ont été créés avec des matrices aléatoires ainsi qu’avec des matrices connues et définies.

Tel qu’attendu, la majorité du temps a été alloué à l’optimisation des implémentations en Julia et à la revue de littérature. Cette dernière a couvert à la fois la théorie derrière la factorisation et les normes de codage en Julia. Le fait de pouvoir réutiliser des portions de code pour chaque implémentation ainsi que pour les tests a permis de réduire un peu le temps passé sur ces éléments.

7 Références

[1] : Nicholas J. Higham. Accuracy and Stability of Numerical Algorithms. Society for Industrial and Applied Mathematics, second edition, 2002. doi: 10.1137/1.9780898718027. URL <https://epubs.siam.org/doi/abs/10.1137/1.9780898718027>

[Lien vers le dépôt Github du projet](#)